
ADVANCED ROBOTICS PROJECT

WEEK 1: CLASSICAL ROBOTICS

Panna Zsoldos, Tamás Piricz
irob.uni-obuda.hu

11 September 2026
Budapest

ÓBUDA UNIVERSITY
ANTAL BEJ CZY CENTER FOR
INTELLIGENT ROBOTICS



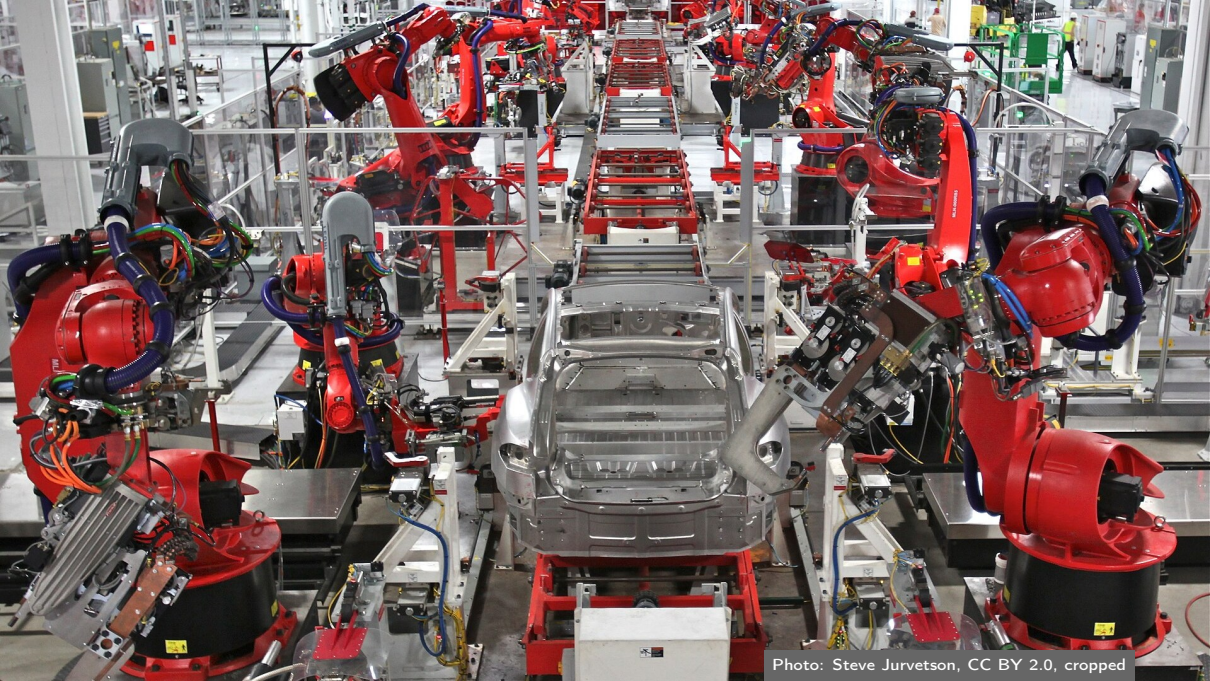
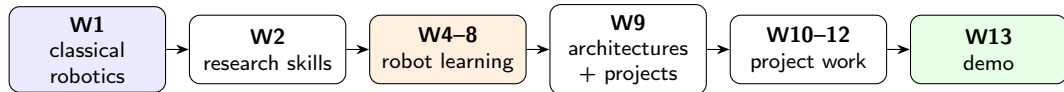


Photo: Steve Jurvetson, CC BY 2.0, cropped



- Today: how robots were made to work **before** machine learning. The vocabulary and the building blocks.
- From week 4: the same problems, solved by **learning from data**.
- This is a practical course. You will build things with different tools, every week.

Week	Date	Topic	Deadline
1	11 Sep	Classical robotics. Semester requirements	H1 out
2	18 Sep	Literature research and scientific skills	
3	25 Sep	<i>no class, rectorial break</i>	
4	02 Oct	Supervised and unsupervised learning	H1 due
5	09 Oct	<i>no class</i>	
6	16 Oct	Reinforcement, imitation, self-supervised learning	H2 out
7	23 Oct	<i>no class, public holiday</i>	
8	30 Oct	Representations in robotics	
9	06 Nov	Modern robot architectures. Project topics	H2 due, projects out
10	13 Nov	Consultation 1	
11	20 Nov	Consultation 2	
12	27 Nov	Consultation 3	project due
13	04 Dec	Project presentations	
14	11 Dec	Retake presentations	

Six contact classes, Fridays from 09:00. Weeks 1, 2 and 9 with Panna; weeks 4, 6 and 8 with Tamás.

What	Requirement	When
Attendance	at least 70% of contact classes, register taken every time	weeks 1, 2, 4, 6, 8, 9
Homework 1	individual, submitted work	out today, due 2 Oct
Homework 2	individual, submitted work	out 16 Oct, due 6 Nov
Semester project	in teams, one of four topics	topics 6 Nov, demo 4 Dec

- Your **homework points** decide the order in which teams choose their project topic.
- The project is graded on three things: the **documentation** (can someone else reproduce it?), a **short video**, and the **live demo**.
- Missed the presentation? One retake in week 14, or in the first week of the exam period by arrangement.

86–100 %	excellent (5)
71–85 %	good (4)
56–70 %	satisfactory (3)
41–55 %	pass (2)
0–40 %	fail (1)

- There is **no exam** and no written test. Everything is coursework.
- Slides and notebooks go to Moodle.
- **Bring your own laptop** to every class. Everything we use runs in a browser and is free.
- Project topics are chosen so that good results can be **written up as a paper**. That is on offer, not required.

- ① **Object manipulation with a Shadow hand**, using Gaussian-splatting models through learning.
- ② **Humanoid torso**: building and bringing it up.
- ③ **Biped robot**: building and bringing it up.
- ④ **Surgical robotics** (da Vinci platform), *to be confirmed*.

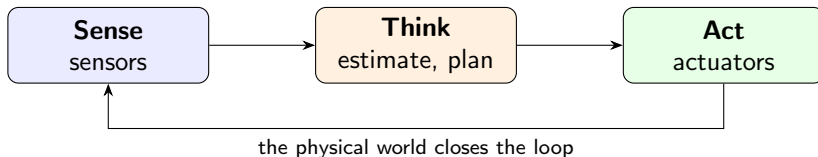
Details and team formation in week 9, after H2. Teams pick in the order of their homework points.

09:05	A robot in the room; rules of the semester
09:20	What is a robot; types and classification
09:35	Lab tour: classify the real ones
09:55	Where is the hand? Frames, forward and inverse kinematics (<i>Notebook 1</i>)
10:35	Break
10:50	How a robot sees: sensors, SLAM
11:02	How a robot drives: a warehouse robot (<i>Notebook 2</i>)
11:32	How behaviour is built; the industry; homework

THE CURRENT STANDARD DEFINITION (ISO 8373:2021)

A robot is a “programmed actuated mechanism with a degree of autonomy to perform locomotion, manipulation or positioning”.

It has been rewritten many times: a reprogrammable manipulator (Robot Institute of America, late 1970s), then ISO 8373 in 1994 (industrial robots only), 2012 and 2021. A new edition is in preparation.



In one sentence: A robot is a closed-loop system that *measures* the physical world and *changes* it. Every topic today is one box in this loop.

Sense

Parts: sensors (cameras, lidar, IMU, encoders) and perception

Fields: electronics, signal processing, computer vision

Think

Parts: estimation and mapping, planning, decisions, learning

Fields: computer science, artificial intelligence, machine learning, probability and statistics

Act

Parts: control, motors and gears, the body (links, joints, wheels, legs)

Fields: control engineering, electrical and mechanical engineering, materials science

Holding it together: software and embedded systems engineering, power electronics and batteries

Around it: human–robot interaction, cognitive science, safety standards, law and ethics

In one sentence: No single field builds a robot. Most hard problems in robotics sit where two of these boxes meet.



KUKA palletizing cell:
the explicit way

EXPLICIT: MODEL THE PHYSICS

Engineers describe the robot and the task with models, then compute what to do.

Classical control, planning, trajectory optimization.

Works best when the physics is well known and the environment is tidy. This is every factory robot today.

IMPLICIT: LEARN IT FROM DATA

Show the robot examples, or let it try, and let it work out by itself what to do in each situation (a **policy**).

Reinforcement learning, imitation learning.

Works best when the world is messy: contact, friction, soft objects, clutter.

In one sentence: Today we stay on the left. Weeks 4–8 are on the right. Same robots, same problems, different tools.

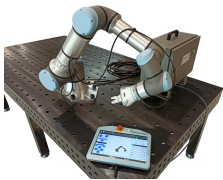
Mainly the world changes	Manipulation	arm, gripper, tool use
Mainly the robot moves	Locomotion	wheeled, legged, aerial
Both	Mobile manipulation	mobile base + arm, humanoid

Try it: a robot opens a door. What changes?

In one sentence: This one question tells you what to sense, what the robot can do, and how to measure success.

Structure	Example	What is easy, what is hard
Serial chain	UR5, Franka	knowing where the hand is: easy
Parallel	Stewart platform, Delta	the opposite
Tree / branched	humanoid, quadruped	many chains share one body
Wheeled	differential drive, car-like	cannot move sideways
Legged	biped, quadruped (GO2)	contact comes and goes
Aerial	multirotor	never at rest

In one sentence: The mechanical structure decides which problems are cheap. That is why factories standardised on 6-axis arms, and why walking robots took thirty years longer.



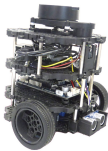
Serial chain: UR16e arm



Parallel: ABB FlexPicker delta robot



Tree: Atlas humanoid



Wheeled: TurtleBot3, differential drive



Legged: ANYmal quadruped

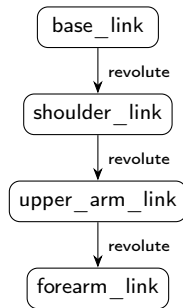


Aerial: quadcopter drone

- **Degree of freedom (DoF)**: one independent way the system can move.
- A free object in space has **6**: move along three axes, rotate about three axes.
- A UR5 arm has **6 joints**: enough to set the tool's position and its orientation together, but not every orientation is possible everywhere. Near the edge of its reach, the choice shrinks.
- Ask for the tool *position* only, and 3 joints' worth of freedom is left over: the arm is **redundant**.

In one sentence: Redundancy is not waste. It is the freedom to reach the same point while avoiding an obstacle, a joint limit, or an awkward pose.

- **Link:** a rigid piece of the robot.
- **Joint:** the connection that lets two links move relative to each other. Revolute (turns), prismatic (slides), fixed.
- **Frame:** a coordinate system glued to a link. “Where is this piece, and which way is it facing?”
- **URDF:** the text file that lists all of this, as a **kinematic tree**: every link except the base has exactly one parent.



In one sentence: Every number a robot reports (joint angle, sensor reading, detected object) lives in *some* frame. Half of practical robotics is knowing which one.

FIRST THE TOUR, THEN THE TASK

We show you around the lab: arm, quadruped, humanoid, mobile base. Then, in groups of two or three, one robot per group: look, touch where allowed, ask.

For your robot, write down:

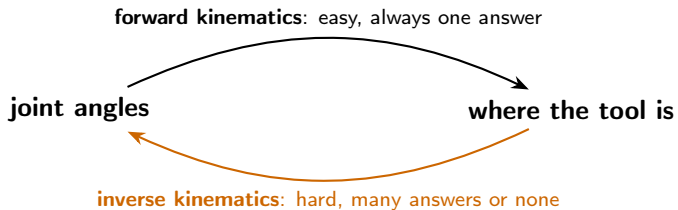
- ① how many links and joints you can find, and what kind of joints,
- ② its degrees of freedom, and whether the task usually leaves some over,
- ③ the shape of its kinematic tree: one chain, or several branches from one body,
- ④ the “what changes?” category, and one thing it clearly *cannot* do.

At the end: one sentence per group.

sense–think–act manipulation locomotion

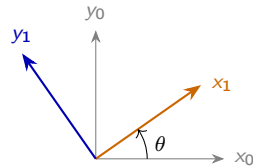
link joint frame URDF

degree of freedom redundancy



- The motors and encoders only know **joint angles**.
- The task is always given as **where the tool should be**.
- Kinematics is the translation between these two languages.

- **Position:** where the point is. Three numbers.
- **Orientation:** which way it faces. Three more.
- Together: a **pose**. Six numbers, one rigid object.
- A **rotation** is stored as a small matrix. Its columns are simply the new axes drawn in the old frame. All rotations together are called **SO(2)** in the plane, **SO(3)** in space (special orthogonal group).

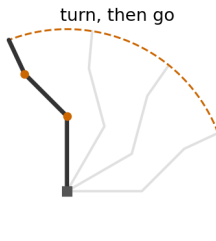
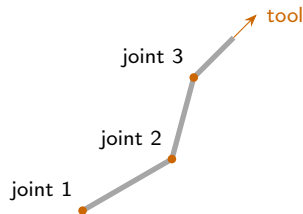


In one sentence: A rotation matrix cannot stretch or squash anything. That is the mathematical way of saying “rigid body”. And undoing a rotation costs nothing.

- The same object has different coordinates in different frames. Nothing moved, only the observer.
- A **transform** is the recipe to move a measurement from one frame to another: a rotation and a shift, packed together.
- Robots pack rotation and shift into one matrix (the **homogeneous transform**) so that changing frames is a single multiplication. All poses together are called **SE(2)** in the plane, **SE(3)** in space (special Euclidean group). Notebook 1 uses these names.

$T_{\text{base_camera}}$: where the camera is, seen from the base. The same matrix turns a point measured by the camera into base coordinates.

In one sentence: In most robot code, read T_{A_B} as “B, as seen from A”, but check each library’s convention. Getting this backwards is one of the most common bugs in robotics.



go, then turn



joint 1 = 90°

[play in the browser](#)

- Walk from the base outwards: at each joint, “turn by this angle, then go along the link in the turned direction”.
- Multiplying these steps, from the base outwards, gives the tool pose. That is all forward kinematics is.
- **Order matters:** turn-then-go is not go-then-turn. Try it with your hand. Right: the same arm simulated both ways.

In one sentence: Forward kinematics: walk along the arm, one turn-then-go step per joint.

- In 3D, getting from one joint frame to the next takes 6 numbers in general. The DH convention places the frames so that **4 per joint** are enough:
- θ , **joint angle**: turn about the joint axis z . This is the number that changes when the joint moves.
- d , **offset** along that z ; then a , **link length** along the new x .
- α , **twist** about that x , lining z up with the next joint axis.
- Careful: datasheets use *standard* or *modified* DH.

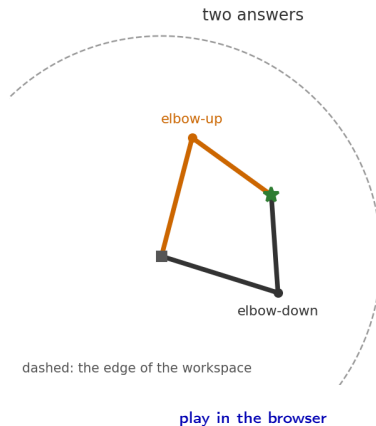
UR5, standard DH

joint	θ	d [m]	a [m]	α
1	q_1	0.089	0	90°
2	q_2	0	-0.425	0°
3	q_3	0	-0.392	0°
4	q_4	0.109	0	90°
5	q_5	0.095	0	-90°
6	q_6	0.082	0	0°

rounded to mm; Notebook 1, Part C uses this table

In one sentence: Four numbers per joint, one row per joint: the whole arm fits in a small table. Each row is one “turn, then go” step, now in 3D.

- 1 **Several answers.** Elbow-up and elbow-down reach the same point. A 6-axis arm typically has eight. The controller has to *choose*.
- 2 **No answer.** Outside the **workspace** there is nothing to compute. Reach has a boundary.
- 3 **Sometimes no formula.** A *closed-form solution* is a direct formula, like the one for the two-link arm in Notebook 1. Arm makers often design their arms so that one exists (the UR5 has one). For a general arm there is none, and the answer is found numerically, by nudging.



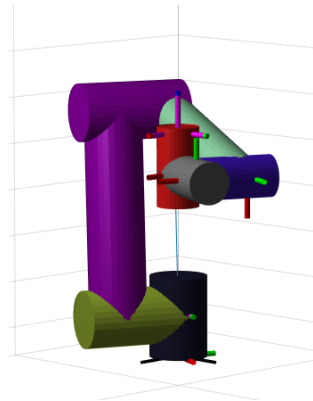
- Move *one* joint a tiny bit. The tool moves a tiny bit in some direction.
- The **Jacobian** is the table of those directions, one column per joint.
- With it you can ask: “I want the tool to go *there*, how should the joints move?”
- In some poses, no combination of joint moves can push the tool in a certain direction: that direction is missing from the table. Such a pose is a **singularity**.

In one sentence: Stretch your arm out fully and try to move your hand further away. You cannot. Your elbow is at a singularity.

- ① Look where the tool is now. Measure the gap to the target.
- ② Use the Jacobian to turn that gap into a small joint move.
- ③ Take the step. Repeat until the gap is tiny.

The **solver** is this whole loop. The **pseudo-inverse** is the tool used in step 2: it turns the gap into a joint move, and when many joint moves would do, it picks the **smallest** one.

Run the same loop once per control cycle, and the solver *is* the controller.



the solver, one frame per step [play in the browser](#)

OPEN IN COLAB, 30 MINUTES

Notebook 1 - Forward and Inverse Kinematics

Open in Colab, or find it at github.com/ABC-iRobotics/advanced-robotics-project-course

- **A:** frames and transforms, forward kinematics of a planar arm, with sliders.
- **B:** inverse kinematics of a two-link arm, both elbows, the workspace boundary.
- **C:** a real UR5 in 3D, the Jacobian, IK by nudging, then closing the loop. *We run this one together and watch.*

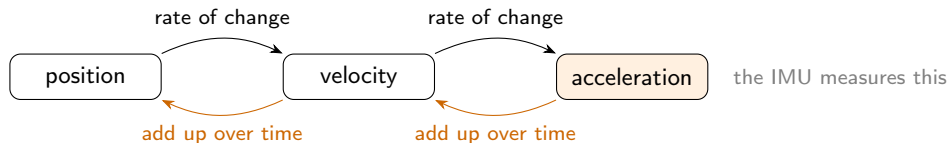
This is where the matrices live. A and B have four one-line exercises, each followed by a check cell that tells you whether it is right. We fill them in together.

pose frame transform DH parameters
forward kinematics inverse kinematics workspace
Jacobian singularity pseudo-inverse

	About itself <i>proprioceptive</i>	About the world <i>exteroceptive</i>
Examples	joint encoders, IMU, joint torque sensors	camera, depth camera, lidar, tactile, GPS
Speed	very fast	slower
Character	precise, but <i>drifts</i> when integrated	noisy, but <i>anchored to the world</i>

In one sentence: The two families fail in opposite ways. That is exactly why robots combine them: this is called **sensor fusion**.

- An **IMU** measures how fast the robot is *accelerating* and *turning*. Not where it is.
- **Velocity** is how fast position changes; **acceleration** is how fast velocity changes. Going back means adding up over time (**integrating**): acceleration gives velocity, velocity gives position. Two sums in a row, so a tiny constant error grows with time squared.
- After a few seconds of “IMU only”, the position estimate is fiction.

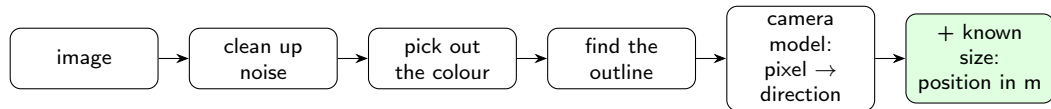


Video: [Pure IMU-based positional tracking is a no-go](#) (Oliver Kreylos, 5 min). The position estimate floats away from 2:15.

In one sentence: Something has to look at the world and pull the drifting estimate back: a lidar, a camera, a GPS. Sensor fusion is not optional.

- Lidar and time-of-flight cameras: send light out, time how long it takes to come back, get a distance, thousands of times per second. Stereo and structured-light depth cameras triangulate instead, from two views or a projected pattern.
- The result is a **point cloud**: a cloud of 3D points, *as seen from the sensor*.
- To use it, the robot has to move the points into its own frame. That is the transform from the previous section.

In one sentence: A sensor never tells you where the wall is. It tells you where the wall is *relative to the sensor*. The frames do the rest.



- **Clean up noise:** classical image filters, for example a Gaussian or a median filter, blur away single wrong pixels. **Camera model:** turns a pixel into a direction from the camera; a known object size then gives the distance.
- Every step is a hand-written rule. Works brilliantly under factory lighting, falls apart when the world changes.

In one sentence: One camera gives you a *direction*, not a distance. Depth needs a second view, a depth sensor, or a known object size.

Localization	the map is known, find the robot on it
Mapping	the robot's path is known, draw the map
SLAM	neither is known: do both at the same time

- While driving, the uncertainty about “where am I” **grows**.
- Seeing a known landmark makes it **shrink**.
- Recognising a place you have visited before is a **loop closure**: the accumulated error snaps back.

Video: [LIO-SAM lidar mapping](#) (Tixiao Shan, 9 min). Doubled walls from drift at 3:40, then “A loop is closed.” at 3:45.

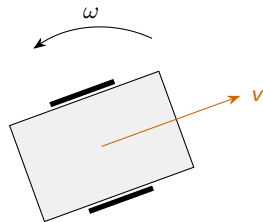
In one sentence: SLAM is a chicken-and-egg problem. It is solved by demanding that all measurements agree with each other at once.

proprioceptive exteroceptive sensor fusion

IMU drift point cloud depth

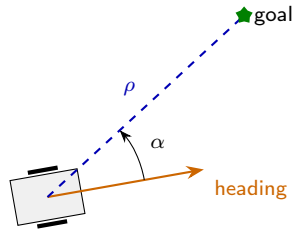
localization mapping SLAM loop closure

- Two driven wheels, one on each side. Same speed: straight. Different speeds: it turns.
- The robot's **state**: where it is and which way it faces, (x, y, θ) .
- What we command: forward speed v and turning rate ω .
- It **cannot move sideways**. Robots with this limitation are called **nonholonomic**.



In one sentence: It can reach any pose, just not directly. Parallel parking is the everyday proof.

- Two things the robot can measure: **how far** the goal is (the **distance error** ρ), and **how far off** its heading is (the **heading error** α).
- Two rules: drive faster when the goal is far, turn harder when the heading is off.
- That is **proportional feedback**: the correction is proportional to the error.

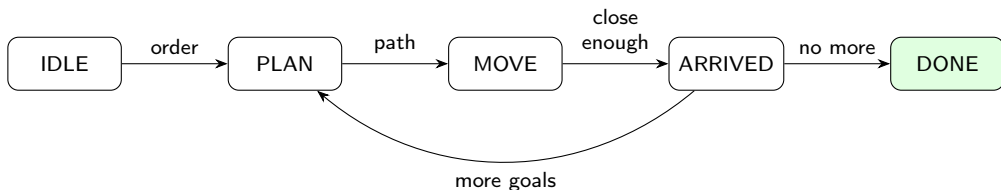


One classic bug: if the heading error is not wrapped to $\pm 180^\circ$, the robot turns the long way round.

In one sentence: Feedback means: measure the error, push against it, repeat. The arm controller in Notebook 1 did exactly the same thing.

- Cut the floor into a grid of cells: free or blocked. Grow the blocked cells by the robot's size, so the robot can be treated as a point.
- **A*** searches the grid for the cheapest path, guided by a guess of the remaining distance.
- The output is a list of **waypoints**. No speeds, no timing.
- Other planners you will meet: Dijkstra, D* Lite (fast replanning), Theta* (any-angle paths), Hybrid A* (car-like robots), and the sampling-based PRM, RRT and RRT*.

In one sentence: The planner answers a geometric question (which way), the controller answers a dynamic one (how fast). Keeping them separate is what makes the classical stack manageable.



In one sentence: Behaviour is a sequence of modes. Inside each mode, the continuous controller runs. This is how industrial robots have been programmed for forty years.

OPEN IN COLAB, 25 MINUTES

Notebook 2 - Mobile Robot Navigation

Open in Colab, or find it at github.com/ABC-iRobotics/advanced-robotics-project-course

- A differential-drive robot you can steer.
- The go-to-goal controller.
- A warehouse grid map and A* path planning.
- A state machine that visits a list of goals, animated end to end.

Four short exercises, each followed by a check cell. The animation you produce here is the same kind you will hand in with H1.

differential drive nonholonomic state

feedback proportional control

path planning A^* waypoint state machine

- **Manipulation:** plan a path for the tool, turn it into joint motion with IK, follow it.
- **Locomotion:** decide where the feet go and when they touch the ground.
- **Whole-body control:** move *all* joints at once so that balance, contact and the task are satisfied together, hundreds of times per second.

In one sentence: A legged robot is an arm that manipulates the ground. The difference is that it has to let go at every step, and still not fall.

State machine

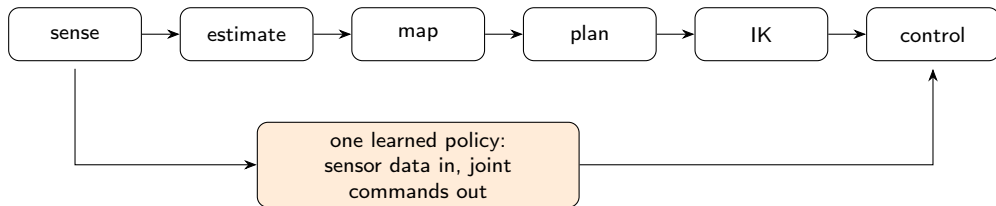
Every new situation adds arrows everywhere.
“What if the battery runs low?” needs an arrow
out of every state.

- **Sequence:** do these in order. **Fallback:** try these until one works. **Retry:** try again.
- A recovery behaviour is one fallback branch, added in one place.
- Nav2, the ROS 2 navigation stack, uses a behaviour tree to decide when to plan, drive and recover.

In one sentence: Same behaviour, better organised. This is an engineering answer, not a mathematical one.

Behaviour tree

A tree of small tasks, checked from the top
many times a second. Each task reports
success, failure, or still running.



Policy: the rule that decides what the robot does next, given what it senses. A *learned* policy is found from data instead of written by hand.

Why people try this: the hand-built boxes are brittle where they meet, they choke on raw camera, touch and sound data, contact and soft objects resist modelling, and there is more and more robot data to learn from.

In one sentence: This is the bridge to weeks 4–8. The learned policy still sends its commands to the same joints. Keep today's vocabulary.

whole-body control

state machine behaviour tree sequence fallback

end-to-end policy

Industrial robots installed worldwide	542 076 robots
Share by region	Asia 74 %, Europe 16 %, Americas 9 %
China's share of these	54 % (295 000 robots)
Chinese vendors' share of their home market	57 % (about 28 % a decade ago)
Robots in operation worldwide (the "stock")	4 664 000 robots
Robots in operation in China	over 2 000 000, five times the US
Installed in Europe	85 000 robots, 8 % fewer than in 2023
Installed in Germany	26 982 robots, 5 % fewer than in 2023

Industrial robots only; one unit is one robot. "Installed" means put into service during 2024. Source: IFR World Robotics 2025, ifr.org.

Established

- KUKA (DE), ABB (CH/SE)
- Universal Robots (DK): the collaborative arm as a product category
- Germany is still the largest European market

New wave

- ANYbotics (ETH spin-off): legged inspection
- Agile Robots (DE): force-controlled manipulation
- Neura Robotics (DE): cognitive robots, humanoids

In one sentence: Europe's strength has been precision, safety and integration. The open question is whether that carries into the learning-driven generation.

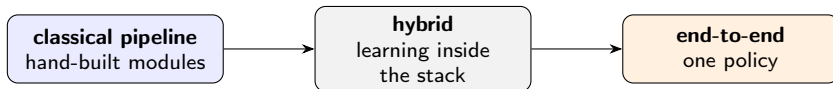
China

- Volume, vertical integration, aggressive pricing
- Unitree, UBTech and others put quadrupeds and humanoids within a university budget
- The GO2 in our lab exists because of this

United States

- Boston Dynamics, Figure, Tesla Optimus
- Strength in software, foundation models and capital

Question for you: where does Europe fit, and what would it take to change that?



- The classical tools do not disappear. They run *inside* modern systems: IK, controllers, planners, state estimators.
- That is why we spent today on them.

SCENARIO

An autonomous mobile robot serves a warehouse. It receives an order: a list of shelves to visit in a given sequence, then it delivers to the packing station.

You get: a template notebook with a new warehouse map, the order, the A* planner, the path follower and the video export.

You write: the state machine for the full cycle, plus *one* realistic complication of your choice:

- ① battery level, with a detour to the charging station,
- ② a blocked aisle discovered on the way, requiring replanning,
- ③ several orders, sequenced by travel distance.

Submit: the notebook, the generated animation as a video, and a one-page description. **Deadline: week 4, 2 October.**

LITERATURE RESEARCH AND SCIENTIFIC METHODS

How to find, read, filter and cite work in robotics, and how to turn a vague interest into a researchable question.

Bring one topic you would like to work on. We will search it together, in class.

Slides and notebooks: Moodle and github.com/ABC-iRobotics/advanced-robotics-project-course.

Questions: after class or by email.



Thank you for attention!

Shown only if there is time, or on request.

3 MINUTES, IN THE BROWSER, NOTHING INSTALLED

A real self-driving car recording (nuScenes) in the Rerun viewer: lidar, radars, six cameras and the car's pose, all on one timeline.

app.rerun.io: [nuScenes example](#)

Watch for:

- the boxes sit on the cars in the camera images only because every frame and transform is right,
- the sensors arrive at completely different rates: lidar 20 times a second, cameras 12, labels 2,
- no drift here: the car's pose was already corrected against a prebuilt map. Raw odometry would drift.

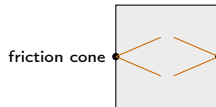
DEMO, 2 MINUTES

A physics simulator (MuJoCo) and a trained policy, both running inside one browser tab: a Unitree G1 humanoid following motions.

motion-tracking.axell.top backup: php-parkour.github.io/demo.html

- MuJoCo is a physics engine many research labs train in.
- The policy outputs joint targets, exactly where our IK output went.
- Drag the robot with the mouse: it recovers, because the policy keeps correcting.
- Nothing was installed. Your project demos can look like this.

- Two fingers, two contact points, facing each other: an **antipodal grasp**.
- Each finger can only push within a cone set by friction. Outside it, the object slips.
- If the fingers can resist any push or twist, the grasp has **force closure**. It will hold.



In one sentence: Classical grasping needs the object's shape and its friction. In real life you know neither. That is why modern grasping is learned from images.

- One robot becomes a *system*: who does which task, who yields in the corridor, who charges when, robots from different vendors in one hall.
- The question is no longer “which joint angle” but “which robot, in which order”.
- Success is measured in orders per hour, not millimetres.

In one sentence: At this level the robot is a resource. The hard problems become scheduling, but a bad low-level controller still ruins the whole schedule.

Videos

- IMU drift: Oliver Kreylos, “Pure IMU-based Positional Tracking is a No-go”, https://youtu.be/_q_8d0E3tDk
- Loop closure: Tixiao Shan, “LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping”, <https://youtu.be/A0H8Co0RZJU>

Demos

- Recorded sensor data: Rerun web viewer, nuScenes example, <https://app.rerun.io/>
- Humanoid in the browser: <https://motion-tracking.axell.top/>, backup <https://php-parkour.github.io/demo.html>

Definitions and numbers

- Robot definition: ISO 8373:2021 Robotics – Vocabulary, term 3.1, <https://www.iso.org/standard/75539.html>
- Industry numbers: IFR World Robotics 2025, <https://ifr.org/>

All photos are from Wikimedia Commons and used under the licence given.

- Industrial robots on a car body line (slide 2): Steve Jurvetson, CC BY 2.0, cropped.
[https://commons.wikimedia.org/wiki/File:Tesla_Robot_Dance_\(7408451314\).jpg](https://commons.wikimedia.org/wiki/File:Tesla_Robot_Dance_(7408451314).jpg)
- KUKA robots palletizing bread (slide 11): KUKA Roboter GmbH, Bachmann, public domain, cropped.
https://commons.wikimedia.org/wiki/File:Factory_Automation_Robotics_Palettizing_Bread.jpg
- UR16e robot arm (slide 14): Auledas, CC BY-SA 4.0.
https://commons.wikimedia.org/wiki/File:UR16e_robot_arm.png
- ABB FlexPicker delta robot (slide 14): Marc Auledas, CC BY-SA 4.0.
https://commons.wikimedia.org/wiki/File:Robot_delta_FlexPicker_d'ABB.png
- Atlas humanoid, 2013 (slide 14): DARPA, public domain.
https://commons.wikimedia.org/wiki/File:Atlas_frontview_2013.jpg
- TurtleBot3 Burger (slide 14): Kuscu0, CC BY-SA 4.0.
https://commons.wikimedia.org/wiki/File:TurtleBot3_Burger.jpg
- ANYmal quadruped (slide 14): ANYbotics, CC BY-SA 4.0.
https://commons.wikimedia.org/wiki/File:ANYbotics_robot_dog_ANYmal.jpg
- DJI Mavic Air 2 quadcopter (slide 14): C.Stadler/Bwag, CC BY-SA 4.0.
https://commons.wikimedia.org/wiki/File:DJI_-_Drohne_Mavic_Air_2.JPG